Security Assessment Report

Compound Service Patch

April 2026

Prepared for Compound

Table of Contents

Project Summary **3**
- Project Scope 3
- Project Overview 3
- Protocol Overview 4

Assessment Methodology **5**

Threat and Security Overview **6**
- **Findings Summary** **8**
- Severity Matrix 8

Detailed Findings **9**
- High Severity Issues 11
  - H-01. Type conversion truncation prevents unpausing collateral assets with index ≥ 8 11
- Medium Severity Issues 13
  - M-01: `transferBase` Missing `MAX_SUPPORTED_UTILIZATION` Check Allows Utilization Cap Bypass 13
- Low Severity Issues 15
  - L-01: activateCollateral Silently Clears Independently-Set Pauses 15
  - L-02: Deactivated collateral supply and transfer bypass via pause flag desynchronization 17
  - L-03: Lack of configuration check in the `getPackedAssetInternal()` function 19
- Informational Severity Issues 21
  - I-01: buyCollateral allows discounted purchase of deactivated collateral from protocol reserves 21
  - I-02: Comet.sol and ScrollComet.sol inherit deactivatedCollaterals storage with no enforcement 23
  - I-03: Collateral supply accrual runs before cheap revert checks 24
  - I-04: Public liquidation check now pays absorb-only memory cost 25
  - I-05.Type truncation in extended pause view functions 25
  - I-06. Pause check after token transfer 27
  - I-07. Missing pause flag for pausing lender deposits while allowing loan repayments 28

Disclaimer **30**

About Certora **30**

certora

Compound

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Comet PRs	https://github.com/woof-software/comet/pull/294	15d5ab0 up to 0905c	EVM

Project Overview

This document describes the security review of **Comet Patch Service PRs**. The work was undertaken in 4 parts from**:**

- **December 10th, 2025 to December 15th, 2025**
- **February 16th, 2026** to **February 23rd, 2026**.
- **March 30th, 2026** to **March 31st, 2026**
- **May 20th 2026** to **May 21st, 2026**

The following contract list is included in our scope:

- contracts/CometCore.sol
- contracts/CometExt.sol
- contracts/CometExtInterface.sol
- contracts/CometMainInterface.sol
- contracts/CometStorage.sol
- contracts/CometWithExtendedAssetList.sol
- contracts/AssetList.sol
- contracts/CometProxyAdmin.sol

A final review of the combined PR was done on April 27th, 2026:

- https://github.com/woof-software/comet/pull/294

Certora performed a manual audit of scope, the discovered bugs are listed on the following page.

Protocol Overview

This audit covers a set of PRs applied to the Woof Software fork of Compound III (Comet), targeting the feat/service-patch branch. The changes implement fixes and hardening measures recommended by an internal security team assessment, which was prompted by the USDM price feed incident. The core modifications span six areas:

(1) **Extended Pause** (PR 249) — introduces a mechanism that provides fine-grained, role-gated pause controls for selective disabling of specific user flows (supply, withdraw, transfer) without halting the entire market.

(2) **Oracle paralysis prevention** (PR 270) — skipping price feed calls for de-listed collaterals (zero collateral factors) in isBorrowCollateralized, isLiquidatable, and absorbInternal to prevent a broken oracle from paralyzing the entire protocol;

(3) **Collateral deactivation lifecycle** (PR 280) — adding pause-guardian-driven emergency deactivation of collateral assets, blocking supply/transfer and forcing borrowers to unwind or be liquidated;

(4) **Utilization cap** (PR 313) — introducing a MAX_SUPPORTED_UTILIZATION (200%) check in withdrawBase to prevent over-utilization leading to reserve exhaustion and cascading liquidations;

(5) **Lender illiquidity prevention** (PR 316) — returning zero borrow/supply rates when there are no borrows or when reserves are exhausted, preventing phantom index growth in empty markets;

(6) **Test suite overhaul** (PRs 243, 321, 323, 319) — replacing harness-based unit tests with flow-based scenario tests covering supply, withdraw, liquidation, repay, and pause behaviors.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The system is a monolithic lending market where a single proxy (CometWithExtendedAssetList) holds all base token and collateral balances, with an extension delegate (CometExt) handling ERC20 metadata and pause control via delegatecall. Trust boundaries separate three privileged roles: the **governor** (full configuration authority via Configurator, reserve withdrawal, activation), the **pause guardian** (emergency pause and collateral deactivation), and **unprivileged users** (supply, borrow, transfer, liquidation).

Key invariants include: utilization must not exceed 200% after a borrow, deactivated collateral must not be suppliable or transferable, de-listed assets (all factors = 0) must never trigger oracle calls, and assetsIn bitmaps must accurately reflect non-zero collateral balances.

Attack vectors examined include: utilization cap bypass via transferBase borrow creation, deactivation bypass via pause flag clearing , partial-zero factor configurations enabling selective oracle paralysis , unauthorized purchase of seized deactivated collateral via buyCollateral , silent clearing of security pauses during collateral re-activation among other attack vectors.

Manual review covered all Solidity changes across PRs 249, 270, 280, 313, 316, 319, 320 and 321 focusing on CometWithExtendedAssetList.sol (the main protocol logic touched by nearly every PR), CometExt.sol (pause and deactivation controls), AssetList.sol (factor validation), CometStorage.sol (new storage slots), and Configurator.sol (configuration validation gaps). Cross-PR interactions were examined, particularly between the oracle-skip logic (PR 270) and the deactivation lifecycle (PR 280). Test PRs (243, 313, 319, 320, 321) provide scenario-based coverage for supply, liquidation, pause, and repay flows but do not cover the deactivation-pause interaction or the transferBase utilization bypass.

The Extended Pause feature adds governance-controlled circuit breakers using bitmap-based pause flags (uint24) to selectively disable supply, withdraw, and transfer operations per-asset or per-user-type (lenders vs borrowers).

Access control is enforced via the `onlyGovernorOrPauseGuardian` modifier, with idempotency checks preventing redundant state changes.

Manual Review Focus: Bitmap manipulation logic for 24 asset indices, enforcement layer integration in core functions (`supplyInternal`, `withdrawBase`, `withdrawCollateral`, `transferBase`, `transferCollateral`), consistency between pause control (`CometExt.sol`) and enforcement checks (`CometWithExtendedAssetList.sol`), and storage layout safety for proxy upgrades.

Key Findings:
One high-severity issue (H-01): type truncation in per-asset pause functions prevents unpausing assets with index ≥ 8, creating permanent lock conditions.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Fixed
Critical	-	-
High	1	1
Medium	1	1
Low	3	3
Informational	7	3
Total	12	8

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
H-01	Type conversion truncation prevents unpausing collateral assets with index ≥ 8	High	Fixed.
M-01	`transferBase` Missing `MAX_SUPPORTED_UTILIZATION` Check Allows Utilization Cap Bypass	Medium	Fixed
L-01	activateCollateral Silently Clears Independently-Set Pauses	Low	Fixed
L-02	Deactivated collateral supply and transfer bypass via pause flag desynchronization	Low	Fixed
L-03	Lack of configuration check in the `getPackedAssetInternal()` function.	Low	Fixed
I-01	buyCollateral allows discounted purchase of deactivated collateral from protocol reserves	Informational	Acknowledged
I-02	Comet.sol and ScrollComet.sol inherit deactivatedCollaterals storage with no enforcement	Informational	Acknowledged
I-03	Collateral supply accrual runs before cheap revert checks	Informational	Acknowledged

I-04	Public liquidation check now pays absorb-only memory cost	Informational	Fixed
I-05	Type truncation in extended pause view functions	Informational	Fixed
I-06	Pause check after token transfer	Informational	Fixed
I-07	Missing pause flag for pausing lender deposits while allowing loan repayments	Informational	Acknowledged

High Severity Issues

H-01. Type conversion truncation prevents unpausing collateral assets with index ≥ 8

Severity: **High**	Impact: **High**	Likelihood: **Medium**
Files: CometExt.sol	Status: Fixed.	

Description:

A type conversion vulnerability exists in `CometExt.sol` that prevents the unpausing of collateral assets with `index ≥ 8`. The vulnerability affects three pause control functions that govern core protocol operations for individual collateral assets:

1. `pauseCollateralAssetWithdraw()`
2. `pauseCollateralAssetSupply()`
3. `pauseCollateralAssetTransfer()`

Each function contains a flawed status check before modifying the pause state:

```
JavaScript
    if (toBool(uint8(collateralsXxxPauseFlags & (uint24(1) << assetIndex))) == paused)
     revert CollateralAssetOffsetStatusAlreadySet(...);
```

When `assetIndex >= 8`, the bit mask value `(uint24(1) << assetIndex)` exceeds 255. For example, `1 << 8 = 256`.

The `uint8` cast silently truncates this to zero, breaking the logic:

- `Asset 8: (1 << 8) = 256 -> uint8(256) = 0`
- `Asset 9: (1 << 9) = 512 -> uint8(512) = 0`

This creates an asymmetric behavior where:

- Pausing works: `toBool(uint8(0)) == true -> false == true -> no revert`
- Unpausing fails: `toBool(uint8(256)) == false -> false == false -> reverts permanently`

`CometExt` is called via `delegatecall` from `CometWithExtendedAssetList` through its fallback function. When assets 8-23 are paused, they become permanently locked because the unpause operation always reverts. This blocks three critical operations in `CometWithExtendedAssetList`, depending on which of the functionalities for specific assets are paused:

1. withdrawCollateral() - Users cannot withdraw collateral
2. supplyCollateral() - Users cannot add collateral to prevent liquidations
3. transferCollateral() - Users cannot transfer collateral between accounts

Once a collateral asset with `index ≥ 8` is paused, it becomes permanently locked with no recovery mechanism short of a contract upgrade.

Recommendations:

Remove the `uint8` cast from all three functions.

```
JavaScript
- if (toBool(uint8(collateralsWithdrawPauseFlags & (uint24(1) << assetIndex))) == paused)

+ if (((collateralsWithdrawPauseFlags & (uint24(1) << assetIndex)) != 0) == paused)
```

Apply the same fix to `pauseCollateralAssetSupply()` and `pauseCollateralAssetTransfer()`.
Customer's response: Fixed in 6e2d418

Fix Review: Fix confirmed.

Medium Severity Issues

M-01: `transferBase` Missing `MAX_SUPPORTED_UTILIZATION` Check Allows Utilization Cap Bypass

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: PR#313	Status: Fixed	

Description:

PR 313 introduces a MAX_SUPPORTED_UTILIZATION cap (200%) in withdrawBase to prevent borrows that would push utilization beyond a safe threshold, protecting against illiquidity and reserve exhaustion. However, transferBase can also create borrows when the sender's resulting balance goes negative, and it lacks this check entirely.

This enables a two-step bypass:

1. Alice (with sufficient collateral, zero base position) calls `transferAsset` to transfer base tokens to Bob (a second address she controls). Inside `transferBase`, Alice's `srcBalance` goes negative, creating a borrow. Simultaneously, Bob receives an equivalent internal supply credit. No utilization check is performed, and no tokens leave the contract.

1. Bob calls `withdraw` on the received balance. Since Bob has a positive base balance, `withdrawBase` takes the lender branch (`srcBalance >= 0`), which also has no utilization check. Tokens are transferred out via `doTransferOut`.

The net economic effect is identical to a direct borrow: totalBorrowBase increases, totalSupplyBase is unchanged, and real tokens leave the protocol -- but the utilization cap is never enforced.

Recommendations:

Add the utilization check to transferBase in a way that accounts for the fact that the destination's newly credited supply may be immediately withdrawn. One approach is to compute

utilization as if only the borrow increase occurred (ignoring the destination's supply increase), since that supply is not locked:

```
if (srcBalance < 0) {

    if (isBorrowersTransferPaused()) revert BorrowersTransferPaused();

    if (uint256(-srcBalance) < baseBorrowMin) revert BorrowTooSmall();

    if (!isBorrowCollateralized(src)) revert NotCollateralized();



    uint totalSupplyWithoutDst = presentValueSupply(baseSupplyIndex,
totalSupplyBase - supplyAmount);

    uint totalBorrow_ = presentValueBorrow(baseBorrowIndex, totalBorrowBase);

    if (totalSupplyWithoutDst > 0 && totalBorrow_ * FACTOR_SCALE /
totalSupplyWithoutDst > MAX_SUPPORTED_UTILIZATION)

        revert ExceedsSupportedUtilization();

}
```

The check needs to compute a worst-case utilization that assumes the destination could immediately withdraw their newly credited supply — because they can. Something like:

Customer Response:

Fixed. (PR#313)

Fix Review:

Fix looks good.

Low Severity Issues

L-01: activateCollateral Silently Clears Independently-Set Pauses

Severity: Low	Impact: Low	Likelihood: Low
Files: PR#280	Status: Fixed	

Description:

The activateCollateral function unconditionally clears the supply and transfer bits within the collateralsSupplyPauseFlags and collateralsTransferPauseFlags bitmaps.

Currently, the protocol uses these shared bitmaps to handle both standard collateral deactivation and security-related, per-asset pauses. Because there is no provenance tracking to determine *why* a specific bit was set, the system cannot distinguish between a bit flipped for routine deactivation and a bit flipped due to an active security incident.

Consequently, invoking activateCollateral removes both types of pauses indiscriminately. This creates a risk where activating a collateral could accidentally reopen supply and transfer functionalities for an asset that was intentionally paused due to an unresolved security

Recommendations:

To prevent accidentally overriding emergency security pauses, consider decoupling the collateral activation state from the security pause state.

Maintain distinct bitmaps or mappings for an asset's "active" status versus its "security pause" status. The activateCollateral function should only toggle the activation status, requiring a separate, explicit action to clear a security pause.

Customer Response:

We are aware of this behavior and confirm that it is intentional. The activation and deactivation flows are designed to be symmetric and emit the corresponding pause events for the supply and transfer flags in both directions.

The pause flags in this context are treated as governance-controlled security controls rather than temporary or automated circuit breakers. They are only used in exceptional cases (e.g., the deUSD wind-down) and are not managed by independent automated processes.

Both setting and clearing these pauses can only be performed through governance actions — either via an on-chain governance vote or via the authorized multisig. No single actor can toggle these flags unilaterally. Therefore, calling activateCollateral cannot accidentally override an active security pause; any reactivation necessarily reflects explicit governance or multisig intent that the restrictions are no longer required.

Because this is a governance-restricted and operationally coordinated process, there is no permissionless or adversarial path that would allow an attacker to clear a security pause. Decoupling activation state from pause state would introduce additional storage and operational complexity without providing meaningful security benefits under the current governance model.

Fix PR

Fix Review:

Fix looks good.

L-02: Deactivated collateral supply and transfer bypass via pause flag desynchronization

Severity: Low	Impact: Medium	Likelihood: Low
Files: PR#270	Status: Fixed	

Description:

supplyCollateral and transferCollateral guard against deactivated collateral exclusively through per-asset pause flags (isCollateralAssetSupplyPaused, isCollateralAssetTransferPaused), with no direct check on isCollateralDeactivated. These pause flags can be independently cleared by the governor or pause guardian via pauseCollateralAssetSupply / pauseCollateralAssetTransfer, which perform no validation against deactivatedCollaterals. This creates a desync where isCollateralDeactivated(i) == true while the corresponding pause flag is false.

Once the desync occurs, two exploit paths open up:

- **Supply path — account self-trapping:** A user supplies deactivated collateral via `supplyCollateral`, which sets their `assetsIn` bit for that asset. If the user later borrows (or already has a borrow position), every subsequent call to `isBorrowCollateralized` reverts with `TokenIsDeactivated`. This traps the account: it cannot borrow, withdraw base, or transfer base — all of which rely on `isBorrowCollateralized` succeeding.
- **Transfer path — spreading to** `dst` without any check: `transferCollateral` only calls `isBorrowCollateralized(src)`, never on `dst`. A non-borrowing `dst` receives deactivated collateral, gets the `assetsIn` bit set via `updateAssetsIn`, and becomes trapped the moment they take on a borrow position.
- **Transfer path — non-borrower** `src` bypass: When `src` has no borrow (`principal >= 0`), `isBorrowCollateralized` returns `true` immediately without iterating assets, so the deactivation check is never reached. Non-borrowing accounts can freely transfer deactivated collateral to arbitrary recipients.

Recommendations:

Add an explicit deactivation guard in both functions, independent of pause flags:

```
if (isCollateralDeactivated(offset)) revert TokenIsDeactivated(asset);
```

- In `supplyCollateral`
- In `transferCollateral`

Additionally, prevent pauseCollateralAssetSupply and pauseCollateralAssetTransfer in CometExt from clearing a pause flag while the corresponding collateral is deactivated:

```
if (!paused && isCollateralDeactivated(assetIndex)) revert
TokenIsDeactivated(assetIndex);
```

The first fix is defense-in-depth at the point of action. The second fix eliminates the desync at its root — making it impossible to unpause a deactivated collateral without first calling activateCollateral (which is governor-only).

Customer Response:

Fix PR

Fix Review:

Fix looks good.

L-03: Lack of configuration check in the `getPackedAssetInternal()` function.

Severity: Low	Impact: Low	Likelihood: Low
Files: PR#270	Status: Fixed	

Description:

A valid configuration for an asset is either that `borrowCollateralFactor` and `liquidateCollateralFactor` are both non-zero, or that they are both zero.

If only one parameter is set to 0, this should be reverted as it is an invalid configuration.

Recommendations:

If `borrowCollateralFactor` or `liquidateCollateralFactor` is 0 you should be reverting:

```
if (assetConfig.borrowCollateralFactor == 0 &&
assetConfig.liquidateCollateralFactor == 0) {
    // De-listed: skip validation
} else if (assetConfig.borrowCollateralFactor != 0 &&
assetConfig.liquidateCollateralFactor != 0) {
    // Both non-zero: validate ordering
    if (assetConfig.borrowCollateralFactor >
assetConfig.liquidateCollateralFactor) revert
CometMainInterface.BorrowCFTooLarge();
    if (assetConfig.liquidateCollateralFactor > MAX_COLLATERAL_FACTOR) revert
CometMainInterface.LiquidateCFTooLarge();
} else {
    // One zero, one non-zero: reject
    revert CometMainInterface.BadAssetConfiguration();
}
```

Customer Response:

Fix PR

The proposed validation logic is reasonable; however, it would change the current configuration model that is already in use across deployed markets.

According to documentation (see the Ethereum USDT market), the deUSD asset has been delisted while retaining a non-zero liquidateCollateralFactor (0.01). This reflects our existing operational approach, where an asset can be deactivated for borrowing while still allowing controlled liquidation with a minimal LCF.

We acknowledge that the current asymmetric parameters stem from the configuration model introduced in CometWithExtendedAssetsList. This behavior is already applied in production markets. Enforcing a requirement that both factors must be simultaneously zero would change this model and could interfere with existing markets as well as the planned upgrades and redeployments.

Our priority is to ensure that the current configuration remains compatible across all markets and does not block the future upgrade process. If you confirm that the proposed validation would not interfere with the upgrade, we can consider implementing it.

Otherwise, given that the current behavior is intentional, documented, and already in use, we would suggest reducing the severity of this finding.

Fix Review:

Fix looks good.

Informational Severity Issues

I-01: buyCollateral allows discounted purchase of deactivated collateral from protocol reserves

Files:

PR#280

Description:

After absorbInternal seizes deactivated collateral into protocol reserves, buyCollateral allows anyone to purchase it with no deactivation check. When liquidationFactor is 0 (expected post-Phase-1 for deactivated assets), the discount formula in quoteCollateral becomes:

```
discountFactor = storeFrontPriceFactor * (1e18 - 0) = storeFrontPriceFactor
assetPriceDiscounted = assetPrice * (1e18 - storeFrontPriceFactor)
```

With a high storeFrontPriceFactor (e.g. 0.6e18), buyers acquire deactivated collateral at 40% of market price. This can front-run governance's intended handling of deactivated reserves via withdrawReserves or OTC deals.

Recommendations:

Add a deactivation check in buyCollateral that reverts when the target asset is deactivated, or document that governance must handle deactivated collateral reserves through withdrawReserves before the discounted buyCollateral path becomes exploitable. At minimum, consider whether deactivated collateral reserves should be excluded from the reserves < targetReserves sell condition.

Customer Response:

Acknowledged, The finding assumes that the liquidationFactor of a deactivated asset will be immediately set to 0. In practice, this parameter is governed by on-chain governance and may or may not be updated during the deactivation process. There is no requirement for it to be set to zero, nor for such a change to occur simultaneously with deactivation.

Deactivation only prevents the asset from being used for new borrowing and disables it as active collateral; it does not inherently modify liquidation parameters. Therefore, the discounted pricing

scenario described in the finding depends on a specific governance configuration that is not part of the standard deactivation flow. Governance can manage seized reserves through standard reserve management mechanisms as appropriate.

Additionally, the code comment referencing “phases” and implying a transition to liquidationFactor = 0 was the result of an internal miscommunication. This comment will be updated to reflect the intended governance-controlled behavior and avoid suggesting an automatic parameter change.

I-02: Comet.sol and ScrollComet.sol inherit deactivatedCollaterals storage with no enforcement

Files:

PR#280

Description:

CometStorage defines deactivatedCollaterals at L105, inherited by all Comet variants. However, Comet.sol and ScrollComet.sol have isBorrowCollateralized implementations that contain no deactivation checks — they iterate assetsIn and accumulate liquidity with no isCollateralDeactivated gate. Since CometExt (the shared extension delegate) exposes deactivateCollateral / activateCollateral, calling those functions on a non-extended deployment would set the bit in storage but the core contract would silently ignore it. The only observable effect would be the pause flags set atomically by deactivateCollateral, and even those per-asset pause flags are not checked in Comet.sol's collateral functions.

Recommendations:

If Comet and ScrollComet deployments are not intended to support the deactivation feature, consider restricting deactivateCollateral / activateCollateral in CometExt to revert when called on non-extended variants, or document that these functions must only be invoked on CometWithExtendedAssetList deployments. Otherwise, an operator calling deactivateCollateral on a standard Comet gets a false sense of security.

Customer Response:

Acknowledged, The Comet.sol implementation referenced in this finding is outdated, and ScrollComet is currently in the process of being deprecated.
https://www.tally.xyz/gov/compound/proposal/544?govId=eip155:1:0x309a862bbC1A00e45506cB8A802D1ff10004c8C0

We already have governance proposals prepared to migrate the remaining Comet deployments on the Mantle and Polygon networks to CometWithExtendedAssetsList, which includes the updated handling for collateral configuration.

The service patch addressing this issue will be released only after these outstanding Comet instances have been upgraded, as the fix is intended to apply to the new implementation rather than the deprecated contracts.

I-03: Collateral supply accrual runs before cheap revert checks

Files:

contracts/CometWithExtendedAssetList.sol, supplyCollateral()

comet@PR#340

Description:

supplyCollateral() now intentionally accrues interest, but the current placement does that work before asset validation and collateral pause checks. As a result, calls that would revert for bad assets or CollateralAssetSupplyPaused still pay the cost of touching global accrual state first. That does not change behavior, but it is wasted gas on failing paths.

Recommendations:

Keep accrueInternal() if fresh interest accrual on collateral supply is required, but move it after asset lookup and pause validation so invalid or paused calls fail before paying the accrual cost.

Customer Response:

Acknowledged

I-04: Public liquidation check now pays absorb-only memory cost

Files:

contracts/CometWithExtendedAssetList.sol

comet@PR#345

Description:

isLiquidatable() now routes through a helper that allocates new uint256[](numAssets) even though the public path only returns a boolean and discards the cached prices. That turns an absorb optimization into permanent overhead on every public liquidation check. With up to 24 assets, this means zero-initializing roughly 800 bytes of memory for no behavioral benefit.

Recommendations:

Keep isLiquidatable() on a bool-only path and reserve price caching for an absorb-specific helper. At minimum, move the array allocation below the principal >= 0 early return so obviously safe accounts do not pay for it.

Customer Response:

Fixed PR#345

Fix Review:

Fix looks good.

I-05. Type truncation in extended pause view functions

Description:

View functions in `CometWithExtendedAssetList.sol` use `uint8` casts when checking `extendedPauseFlags` (declared as `uint24`), limiting implementation to 8 bits despite the type system indicating 24 bits are available.

Affected: isCollateralWithdrawPaused(), isCollateralSupplyPaused(), isBaseSupplyPaused(), isLendersTransferPaused(), isBorrowersTransferPaused(), currentPauseOffsetStatus()

```javascript
   function isCollateralWithdrawPaused() public view returns (bool) {
    return toBool(uint8(extendedPauseFlags & (uint24(1) << PAUSE_COLLATERALS_WITHDRAW_OFFSET)));
    //             ^^^^^ Truncates to 8 bits
}
```

Currently safe: All 8 offsets (0-7) are within `uint8` range.

Future risk: If offsets 8-23 are added, checks silently return incorrect results:

```javascript
uint24(1) << 8 = 256 → uint8(256) = 0 → toBool(0) = false
```

Recommendations:

Remove `uint8` casts:

```javascript
function isCollateralWithdrawPaused() public view returns (bool) {
-   return toBool(uint8(extendedPauseFlags & (uint24(1) << PAUSE_COLLATERALS_WITHDRAW_OFFSET)));
+   return (extendedPauseFlags & (uint24(1) << PAUSE_COLLATERALS_WITHDRAW_OFFSET)) != 0;
}
```

Customer's response: Fixed in c1e706a & 41461c0

Fix Review: Fix confirmed.

I-06. Pause check after token transfer

Description:

In the `supplyCollateral()` function, the per-asset pause check is performed AFTER the ERC-20 token transfer, rather than before. While the transaction correctly reverts if paused, this ordering creates unnecessary gas waste for users.

```JavaScript
   function supplyCollateral(address from, address dst, address asset, uint128 amount) internal
{
    amount = safe128(doTransferIn(asset, from, amount));

    AssetInfo memory assetInfo = getAssetInfoByAddress(asset);
    uint8 offset = assetInfo.offset;

@>  if (isCollateralAssetSupplyPaused(offset)) revert CollateralAssetSupplyPaused(offset);
}
```

Other pause enforcement functions correctly check pause status BEFORE external calls:

```JavaScript
function withdrawCollateral(...) internal {
    AssetInfo memory assetInfo = getAssetInfoByAddress(asset);
    uint8 offset = assetInfo.offset;

@>    if (isCollateralAssetWithdrawPaused(offset)) revert CollateralAssetWithdrawPaused(offset);


    doTransferOut(asset, to, amount);
}
```

Recommendations:

Move the pause check before `doTransferIn()`:

```JavaScript
function supplyCollateral(address from, address dst, address asset, uint128 amount) internal {
+   AssetInfo memory assetInfo = getAssetInfoByAddress(asset);
```

```
+   uint8 offset = assetInfo.offset;
+
+   if (isCollateralAssetSupplyPaused(offset)) revert CollateralAssetSupplyPaused(offset);
+
    amount = safe128(doTransferIn(asset, from, amount));

-   AssetInfo memory assetInfo = getAssetInfoByAddress(asset);
-   uint8 offset = assetInfo.offset;
-
-   if (isCollateralAssetSupplyPaused(offset)) revert CollateralAssetSupplyPaused(offset);

    TotalsCollateral memory totals = totalsCollateral[asset];
    ...
}
```

Customer's response: Fixed in 1b39615

Fix Review: Fix confirmed.

I-07. Missing pause flag for pausing lender deposits while allowing loan repayments

Description:

The extended pause system provides separate pause controls for lenders and borrowers in withdrawal and transfer operations, but not for supply. The `isBaseSupplyPaused()` flag blocks both lender deposits and borrower repayments without distinction.

This creates a problematic scenario during market stress: pausing new deposits to limit exposure also prevents borrowers from repaying their loans, when reducing debt is critical.

Recommendations:

Add separate pause flags for lender deposits and borrower repayments.

Customer's response: Acknowledged. This functionality was outside the scope of the audit. Additionally, we do not see a market scenario where pausing base asset lending while still

allowing borrower repayments would be beneficial. Both supplying the base asset and repaying borrowed positions are core protocol operations and are intended to remain available at all times.

As a result, the current behavior of isBaseSupplyPaused(), which affects both deposits and repayments, is considered acceptable within the intended design and threat model of the protocol.

I-08. Missing pause flag for pausing lender deposits while allowing loan repayments

Description:

The extended pause system provides separate pause controls for lenders and borrowers in withdrawal and transfer operations, but not for supply. The `isBaseSupplyPaused()` flag blocks both lender deposits and borrower repayments without distinction.

This creates a problematic scenario during market stress: pausing new deposits to limit exposure also prevents borrowers from repaying their loans, when reducing debt is critical.

Recommendations:

Add separate pause flags for lender deposits and borrower repayments.

Customer's response: Acknowledged. This functionality was outside the scope of the audit. Additionally, we do not see a market scenario where pausing base asset lending while still allowing borrower repayments would be beneficial. Both supplying the base asset and repaying borrowed positions are core protocol operations and are intended to remain available at all times.

As a result, the current behavior of isBaseSupplyPaused(), which affects both deposits and repayments, is considered acceptable within the intended design and threat model of the protocol.

I-09. Different active assets checks allow seizure of assets that don't improve loan health

Description:

During the liquidation flow, the functions isLiquidatableInternal and absorbInternal use different variables to determine whether an asset is delisted - the former checks it by comparing liquidateCollateralFactor to 0 while the latter checks it against liquidationFactor. If the liquidate collateral factor is 0 while the liquidation factor is non-0, an asset wouldn't improve a loan's health, but would be seizable.

Recommendations:

Use a unified check where seizable assets are only those who improve a loan's health.

Certora:

After a discussion with the customer, this functionality is a deliberate choice in the hands of Compound Governance. It is an intended design, not requiring a fix.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.